EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
    - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
    - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
    - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
    - **Extract Transport Header:**
        - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

      If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.
6. **Clean Up:** Close the socket upon receiving a termination signal.

Code Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int size) {
    // Extract IP Header
    struct iphdr *ip_hdr = (struct iphdr *)buffer;
    unsigned int ip_hdr_len = ip_hdr->ihl * 4;

    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = ip_hdr->saddr;

    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = ip_hdr->daddr;

    printf("\n================ Packet Captured
================\n");
```

```
printf("IP Header:\n");
printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
printf("   |-Protocol          : %d\n", ip_hdr->protocol);
printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

// Handle TCP Packets
if (ip_hdr->protocol == IPPROTO_TCP) {
    struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
    printf("TCP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
    printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
    printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
    printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
    printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
    printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
}
// Handle UDP Packets
else if (ip_hdr->protocol == IPPROTO_UDP) {
    struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
    printf("UDP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
```

```
        printf("   |-UDP Length        : %u\n", ntohs(udp_hdr->len));
    }

printf("============================================
=\n");
}

int main() {
    int raw_sock;
    socklen_t saddr_len;
    struct sockaddr saddr;
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);

    if (buffer == NULL) {
        printf("Buffer allocation failed.\n");
        return 1;
    }

    // Create raw socket to capture IP packets
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (raw_sock < 0) {
        perror("Socket creation error (ensure you run with sudo)");
        free(buffer);
        return 1;
    }

    printf("Starting Packet Sniffer...\nPress Ctrl+C to stop.\n");
```

```
    // Capture 3 packets as an example output
    int packet_count = 0;
    while (packet_count < 3) {
        saddr_len = sizeof(saddr);
        int data_size = recvfrom(raw_sock, buffer, BUFFER_SIZE, 0,
&saddr, &saddr_len);

        if (data_size < 0) {
            perror("Recvfrom error");
            free(buffer);
            close(raw_sock);
            return 1;
        }

        process_packet(buffer, data_size);
        packet_count++;
    }

    close(raw_sock);
    free(buffer);
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./sniffer

Output

Terminal Screen:

Starting Packet Sniffer...

Press Ctrl+C to stop.

================ Packet Captured ================

IP Header:

|-IP Version : 4

|-Header Length : 20 Bytes

|-Time To Live (TTL): 64

|-Protocol : 6

|-Source IP : 192.168.1.5

|-Destination IP : 93.184.216.34

TCP Header:

|-Source Port : 49152

|-Destination Port : 80

|-Sequence Number : 10423011

|-Acknowledge No : 0

|-SYN Flag : 1

|-ACK Flag : 0

|-FIN Flag : 0

==